

Proyecto: Censo de Pacientes y Medicamentos de Alto Costo

1. Contexto del Proyecto

Estamos desarrollando un backend con **FastAPI** y **PostgreSQL** (corriendo en Docker) para gestionar el censo de pacientes y la asignación de medicamentos de alto costo. El sistema utiliza un esquema **RBAC** (Control de Acceso Basado en Roles) con tres niveles: `SUPER_ADMIN`, `ADMIN_ESTATAL` y `RESPONSABLE_UNIDAD`.

2. Stack Tecnológico

- **Framework:** FastAPI
- **ORM:** SQLAlchemy (v2.0+)
- **Base de Datos:** PostgreSQL 15 (Docker)
- **Seguridad:** JWT (python-jose) + Passlib (bcrypt)
- **Entorno:** Python 3.13 con venv

3. Estado Actual

- El entorno virtual y Docker están configurados y funcionando.
- Se ha creado con éxito el primer usuario `SUPER_ADMIN` mediante un script manual (`create_admin.py`) que utiliza `bcrypt` directamente.
- Las tablas en la base de datos ya fueron creadas por el script mencionado.
- **El servidor uvicorn corre**, pero falla en el proceso de autenticación web.

4. El Problema Técnico (Bloqueante)

Estamos enfrentando una incompatibilidad conocida entre las versiones recientes de la librería `bcrypt` y `passlib`.

- **Error observado:** `AttributeError: module 'bcrypt' has no attribute '__about__'` y errores de 72 bytes al intentar verificar contraseñas.
- **Impacto:** Al intentar hacer Login en `/auth/login` vía Swagger UI, el servidor devuelve un **Error 500**.
- **Solución temporal aplicada en script:** Se usó un "patch" para inyectar `__about__` en el módulo `bcrypt` o se usó `bcrypt` de forma directa sin pasar por `passlib`.

5. Archivos Clave

- `app/models.py`: Define las clases Usuario, Paciente, Suministro, UnidadMedica y CatMedicamento.
- `app/auth.py`: Contiene la lógica de JWT y la función `verify_password` que está fallando.
- `app/database.py`: Configuración de la conexión a PostgreSQL.
- `create_admin.py`: Script que funcionó para insertar el primer usuario saltándose `passlib`.

6. Sigüientes Pasos Requeridos

1. **Corregir `app/auth.py`**: Aplicar el parche de compatibilidad de `bcrypt` de forma que el servidor no lance el Error 500.
2. **Validar Login**: Lograr obtener el `access_token` desde Swagger UI.
3. **Poblar Catálogos**: Empezar con el registro de Unidades Médicas y Medicamentos.
4. **Validación de RBAC**: Asegurar que los filtros por `clues` e `id_entidad` definidos en el Blueprint se apliquen correctamente en los endpoints de Pacientes.

Nota para el Arquitecto: Por favor, revisa `app/auth.py` y aplica el parche de `bcrypt` antes de proceder con el desarrollo de los módulos de Suministros.